

Programmation fonctionnelle 2
Preuves et programmes
CM4 : `forall` et `False` (encore)

Pierre Rousselin

Université Sorbonne Paris Nord

février 2026

Tactiques et règles de déduction naturelle

connecteur	introduction (prouver)	élimination (utiliser)
$\rightarrow$	intros	apply
$\wedge$	split	destruct
$\vee$	left ou right	destruct
$\forall$	intros	specialize , instantiation ¹
$\perp$		destruct
$\exists$	exists	destruct

Dans chaque cas, il s'agit, en fait :

- ▶ de définition (**intros**) de fonctions ou
- ▶ d'application (**apply**) de fonctions ou
- ▶ d'élimination (**destruct**) d'un terme dont le type est inductif (donc en fait **match**) ou
- ▶ de construction (**split**, **left**, **right**) d'un élément d'un type inductif avec un de ses constructeurs appliqué à des arguments du bon type

1. explicite ou implicite avec unification

Types dépendants d'un type et $\forall$

Prouver `forall`

Utiliser `forall`

Types dépendants de valeurs

`False`, encore

Généricité en OCaml

- Un type non contraint est générique :

```
let id x = x
(* val id : 'a -> 'a = <fun> *)
```

- On peut penser au type de `id` comme :

$$\text{id} : \forall \alpha, \quad \alpha \rightarrow \alpha.$$

- ```
let proj1 x y = x
(* val proj1 : 'a -> 'b -> 'a = <fun> *)
let proj2 x y = y
(* val proj2 : 'a -> 'b -> 'b = <fun> *)
```

- On peut penser au type de `proj1` et `proj2` comme :

$$\text{proj1} : \forall \alpha, \forall \beta, \quad \alpha \rightarrow \beta \rightarrow \alpha$$

$$\text{proj2} : \forall \alpha, \forall \beta, \quad \alpha \rightarrow \beta \rightarrow \beta$$

- La quantification se fait sur les types. Mais il n'y a pas de sorte (type de types) en OCaml.

# Types paramétrés en OCaml

- ▶ Les variants peuvent être paramétrés par des types.
- ▶ `type 'a box = Box of 'a`
- ▶ `let box1 = Box 42`  
*(\* int box = Box 1 \*)*
- ▶ On peut voir le type `box` comme associant à tout type  $\alpha$ , un type  $(\alpha \text{ box})$ , donc une fonction des types dans les types.
- ▶ Le type `box` n'est pas très utile.
- ▶ Celui-ci l'est beaucoup :  
`type 'a option = None | Some of 'a`
- ▶ Permet de représenter une valeur ou l'absence de valeur sans lever d'exception
- ▶ et sans utiliser de valeur particulière du type (comme `NULL` ou `-1`).
- ▶ `let some x = Some x`  
*(\* val some : 'a -> 'a option = <fun> \*)*
- ▶ Moralement, le type de `some` est

`some :  $\forall \alpha, \alpha \rightarrow \alpha \text{ option}$`

# Types paramétrés en Rocq

- ▶ Pas de paramètres implicites de types, et on a les sortes (types de types).
- ▶ **Inductive** box (A : **Type**) := Box (a : A).  
**Check** box. (\* box : Type -> Type \*)  
**Check** Box. (\* Box : forall A : Type, A -> box A \*)  
**Check** (Box nat). (\* Box nat : nat -> box nat \*)  
**Check** (Box bool). (\* Box bool : bool -> box bool \*)  
**Check** (Box bool true). (\* Box bool true : box bool \*)
- ▶ Le constructeur Box est ici une vraie fonction, dont le type est  
**Check** Box. (\* Box : forall A : Type, A -> box A \*)
- ▶ C'est un type **dépendant** (ici d'un autre type).
- ▶ Le mot-clé primitif **forall** sert à signifier cette dépendance.
- ▶ Box est bien une fonction, mais d'un type particulier :
  - ▶ le premier paramètre est un type A
  - ▶ le deuxième paramètre est un élément de A, donc **le type du deuxième argument dépend du premier.**

# Autres types dépendants

- ▶ Type produit (déjà vu) :  
`Inductive prod (A B : Type) := pair (a : A) (b : B).`
- ▶ Type de `prod` ?

# Autres types dépendants

- ▶ Type produit (déjà vu) :

```
Inductive prod (A B : Type) := pair (a : A) (b : B).
```

- ▶ Type de prod?

```
Check prod. (* prod : Type -> Type -> Type *)
```

- ▶ Type de pair?



# Autres types dépendants

- ▶ Type produit (déjà vu) :

```
Inductive prod (A B : Type) := pair (a : A) (b : B).
```

- ▶ Type de prod?

```
Check prod. (* prod : Type -> Type -> Type *)
```

- ▶ Type de pair?

```
Check pair. (* pair : forall (A B : Type), A -> B -> prod A B *)
```

- ▶ Type union (auss appelé « type somme ») :

```
Inductive sum (A B : Type) := inl (a : A) | inr (b : B).
```

- ▶ Type de sum, de inl, de inr?

# Autres types dépendants

- ▶ Type produit (déjà vu) :

```
Inductive prod (A B : Type) := pair (a : A) (b : B).
```

- ▶ Type de prod?

```
Check prod. (* prod : Type -> Type -> Type *)
```

- ▶ Type de pair?

```
Check pair. (* pair : forall (A B : Type), A -> B -> prod A B *)
```

- ▶ Type union (auss appelé « type somme ») :

```
Inductive sum (A B : Type) := inl (a : A) | inr (b : B).
```

- ▶ Type de sum, de inl, de inr?

```
Check sum. (* sum : Type -> Type -> Type *)
```

```
Check inl. (* inl : forall (A B : Type), A -> sum A B *)
```

```
Check inr. (* inr : forall (A B : Type), B -> sum A B *)
```

## Même chose dans Prop

- ▶ pareil, sauf que Prop est la sorte des propositions
- ▶ Équivalent de prod :  
`Inductive and (A B : Prop) : Prop := conj (hA : A) (hB : B).`
- ▶ Type de and, de conj ?

## Même chose dans Prop

- ▶ pareil, sauf que Prop est la sorte des propositions
- ▶ Équivalent de prod :  
`Inductive and (A B : Prop) : Prop := conj (hA : A) (hB : B).`
- ▶ Type de and, de conj ?  
`Check and. (* and : Prop -> Prop -> Prop *)`  
`Check conj. (* conj : forall (A B : Prop), A -> B -> and A B *)`
- ▶ Équivalent de sum :  
`Inductive or (A B : Prop) : Prop :=  
 or_introl (hA : A) | or_intror (hB : B).`
- ▶ Type de or, de or\_introl, de or\_intror ?

## Même chose dans Prop

- ▶ pareil, sauf que Prop est la sorte des propositions

- ▶ Équivalent de prod :

**Inductive** and (A B : Prop) : Prop := conj (hA : A) (hB : B).

- ▶ Type de and, de conj ?

**Check** and. (\* and : Prop -> Prop -> Prop \*)

**Check** conj. (\* conj : forall (A B : Prop), A -> B -> and A B \*)

- ▶ Équivalent de sum :

**Inductive** or (A B : Prop) : Prop :=  
or\_introl (hA : A) | or\_intror (hB : B).

- ▶ Type de or, de or\_introl, de or\_intror ?

**Check** or. (\* or : Prop -> Prop -> Prop \*)

**Check** or\_introl. (\* or\_introl : forall (A B : Prop), A -> or A B \*)

**Check** or\_intror. (\* or\_intror : forall (A B : Prop), B -> or A B \*)

Types dépendants d'un type et  $\forall$

Prouver **forall**

Utiliser **forall**

Types dépendants de valeurs

**False**, encore

# Prouver forall

- ▶ Prouver un énoncé commençant par `forall (A : Prop)` (par exemple) revient à construire un terme
- ▶ dont le type commence par `forall (A : Prop)`
- ▶ donc une **fonction** qui a un type **dépendant**.

```
Lemma foo1 : forall (A : Prop), A -> A.
```

```
Proof.
```

```
 exact (fun (A : Prop) => fun (hA : A) => hA).
```

```
Qed.
```

- ▶ Idem avec sucre syntaxique.

```
Lemma foo2 : forall (A : Prop), A -> A.
```

```
Proof.
```

```
 exact (fun (A : Prop) (hA : A) => hA).
```

```
Qed.
```

## forall avant ce cours

- ▶ Au passage, pendant les premières semaines, on a plutôt écrit

```
Lemma foo3 (A : Prop) : A -> A.
```

```
Proof. exact (fun (hA : A) => hA). Qed.
```

- ▶ voire

```
Lemma foo4 (A : Prop) (hA : A) : A. Proof. exact hA. Qed.
```

- ▶ foo3 et foo4 sont en fait identiques à foo1 et foo2.

```
foo1 = fun (A : Prop) (hA : A) => hA
```

```
 : forall A : Prop, A -> A
```

```
foo2 = fun (A : Prop) (hA : A) => hA
```

```
 : forall A : Prop, A -> A
```

```
foo3 = fun (A : Prop) (hA : A) => hA
```

```
 : forall A : Prop, A -> A
```

```
foo4 = fun (A : Prop) (hA : A) => hA
```

```
 : forall A : Prop, A -> A
```

- ▶ En fait les arguments des lemmes ou des définitions qui apparaissent avant : sont du sucre syntaxique pour

- ▶ `forall` (truc : TypeDeTruc) dans le type

- ▶ `fun` (truc : TypeDeTruc) dans la valeur



## forall et $\rightarrow$

- ▶ Tant qu'à lever le voile, autant y aller à fond...
- ▶ Dans  
`Lemma foo4 (A : Prop) (hA : A) : A. Proof. exact hA. Qed.`
- ▶ Comment Rocq fait la différence dans le type entre `forall (A : Prop)` et le premier `A` de `A  $\rightarrow$  A` ?

## forall et ->

- ▶ Tant qu'à lever le voile, autant y aller à fond...
- ▶ Dans  
`Lemma foo4 (A : Prop) (hA : A) : A. Proof. exact hA. Qed.`
- ▶ Comment Rocq fait la différence dans le type entre `forall (A : Prop)` et le premier `A` de `A -> A` ?
- ▶ Il ne fait pas de différence.
- ▶ La flèche `A -> B` en Rocq n'est qu'une notation pour le type `forall (a : A), B`,
- ▶ le type des fonctions qui pour chaque élément `a` de type `A`, retourne un élément de type `B`.
- ▶ Comme ce type n'est pas dépendant (`a` n'apparaît pas dans ce qui est derrière `forall`),
- ▶ Rocq l'affiche sous la forme `A -> B`.
- ▶ `Notation "A -> B" := (forall _ : A, B) : type_scope.`

# Prouver forall avec une tactique

- ▶ Comme pour les implications (fonctions de types non dépendants)
- ▶ l'équivalent de `fun x =>` est `intros x`.
- ▶ `Lemma foo5 : forall (A : Prop), A -> A.`

`Proof.`

`intros A. (* Soit A une proposition. *)`

`intros hA. (* On suppose hA : A *)`

`exact hA.`

`Qed.`

# Prouver forall avec une tactique

- ▶ Comme pour les implications (fonctions de types non dépendants)
- ▶ l'équivalent de `fun x => est intros x.`

▶ `Lemma foo5 : forall (A : Prop), A -> A.`

`Proof.`

`intros A. (* Soit A une proposition. *)`

`intros hA. (* On suppose hA : A *)`

`exact hA.`

`Qed.`

- ▶ On peut même `intros` plusieurs variables (hypothèses) d'un seul coup.

`Lemma foo6 : forall (A : Prop), A -> A.`

`Proof.`

`intros A hA.`

`exact hA.`

`Qed.`

# Introduction du « pour tout »

Règle d'introduction du « pour tout » en déduction naturelle.

$$\frac{\Gamma, x : T \vdash P}{\Gamma \vdash \forall x : T, P} (\forall I)$$

- ▶ De haut en bas : si dans le contexte  $\Gamma$  avec en plus l'hypothèse que  $x$  est de type  $T$  (et aucune autre hypothèse sur  $x$ ), je sais prouver  $P$  ; alors dans le contexte  $\Gamma$ , je sais prouver  $\forall x : T, P$ .
- ▶ Et de bas en haut, pour prouver à partir de  $\Gamma$  la formule  $\forall x : T, P$ , il suffit de prouver  $P$  à partir de  $\Gamma$  auquel on ajoute l'hypothèse  $x : T$  (et aucune autre hypothèse sur  $x$ ).

Types dépendants d'un type et  $\forall$

Prouver `forall`

Utiliser `forall`

Types dépendants de valeurs

`False`, encore

# Application de fonction

- ▶ On peut utiliser un énoncé de type `forall` comme n'importe quelle fonction.
- ▶ `Lemma impl_refl : forall (A : Prop), A -> A.`  
`Proof. intros A hA. exact hA. Qed.`  
`Lemma bar1 : forall (A B : Prop), (A -> B -> A) -> (A -> B -> A).`  
`Proof.`  
    `intros A B.`  
    `exact (impl_refl (A -> B -> A)).`  
`Qed.`
- ▶ `Check or_introl. (* or_introl : forall (A B : Prop), A -> A \ / B *)`  
`Lemma bar2 : forall A : Prop, A -> A \ / A.`  
`Proof.`  
    `intros A hA.`  
    *(\* point technique : or\_introl a ses premiers arguments*  
    *\* qui sont implicites \*)*  
    *(\* le @ avant permet de récupérer la version sans implicites \*)*  
    `exact (@or_introl A A hA).`  
`Qed.`

# La tactique `specialize`

- ▶ On peut aussi utiliser la tactique `specialize` pour
  - ▶ ajouter un terme du contexte global dans le contexte local (jamais indispensable mais peut être utile)
  - ▶ donner des arguments à une fonction, c'est-à-dire
  - ▶ donner une version spécialisée d'un énoncé.

▶ `Lemma bar3 : forall (A B : Prop), (A -> B -> A) -> (A -> B -> A).`

`Proof.`

`intros A B.`

`specialize impl_refl as H.`

`(* H : forall A : Prop, A -> A *)`

`specialize (H (A -> B -> A)).`

`(* H : (A -> B -> A) -> A -> B -> A *)`

`exact H.`

`Qed.`



## La tactique specialize (2)

- `Lemma bar4 : forall A : Prop, A -> A \/\ A.`

`Proof.`

`intros A hA.`

`specialize or_introl as oinl.`

`(* oinl : forall A B : Prop, A -> A \/\ B *)`

`specialize (oinl A A) as H.`

`(* H : A -> A \/\ A *)`

`specialize (H hA).`

`(* H : A \/\ A *)`

`exact H.`

`Qed.`

- Sans `as`, sur un terme `h` du contexte local, remplace `h` par sa version spécialisée.
- La version avec `as` est non destructive, et permet d'utiliser un *intro pattern* derrière `as`.

# Élimination du « pour tout »

Règle d'élimination du « pour tout » en déduction naturelle (par exemple, une version typée).

$$\frac{\Gamma \vdash \forall(x : T), P \quad \Gamma \vdash t : T}{\Gamma \vdash P\{x/t\}} (\forall\text{I})$$

où  $P\{x/t\}$  désigne le terme obtenu en remplaçant (les occurrences libres de)  $x$  par le terme  $t$ .

Types dépendants d'un type et  $\forall$

Prouver `forall`

Utiliser `forall`

Types dépendants de valeurs

`False`, encore

# Motivation

- ▶ On sait quantifier universellement un énoncé sur des types, donc en particulier des propositions.
- ▶ Comment parler d'un énoncé qui porterait, par exemple,
  - ▶ sur tous les booléens
  - ▶ sur tous les entiers naturels
- ▶ Contrairement à OCaml, la théorie des types de Rocq est une théorie des types dépendants.
- ▶ On peut donc concevoir une fonction qui prend un booléen `b`
- ▶ et retourne, par exemple,
  - ▶ un `bool` si `b` vaut `true`
  - ▶ un `nat` si `b` vaut `false`

# Première tentative

- ▶ **Fail Definition** `false_or_42 (b : bool) :=  
 match b with  
 | true => false  
 | false => 42  
end.`
- ▶ The command has indeed failed with message:  
The term "42" has type "nat" while it is expected  
to have type "bool".
- ▶ Comment typer cette fonction ?

## match dépendant

- ▶ Rocq ne peut pas à la fois :
  - ▶ inférer le type d'une expression `match`
  - ▶ détecter les éventuelles erreurs de type et
  - ▶ permettre aux branches `match` d'avoir des types différents.
- ▶ On doit indiquer avec une clause `return` le `type commun` des branches.

- ▶ Ici :

```
Definition false_or_42 (b : bool) :=
 match b return (if b then bool else nat) with
 | true => false
 | false => 42
end.
```

- ▶ Le type de la fonction est :

```
false_or_42 : forall b : bool, if b then bool else nat
```

# Propositions universellement quantifiées

- ▶ On obtient alors un système de types qui permet d'exprimer des propositions universellement quantifiées sur n'importe quel type.
- ▶ Des exemples (certains déjà vus, d'autres qu'on reverra ultérieurement) :

- ▶ `Lemma and_true_1 : forall (b : bool), andb true b = b.`  
`Proof. intros b. reflexivity. Qed.`  
`Lemma add_0_l : forall (n : nat), 0 + n = n.`  
`Proof. intros n. reflexivity. Qed.`  
`Lemma le_refl : forall (n : nat), n <= S n.`  
`Proof. intros n. apply le_S. exact (le_n n). Qed.`

Types dépendants d'un type et  $\forall$

Prouver `forall`

Utiliser `forall`

Types dépendants de valeurs

**False**, encore



# Règle d'élimination de False

- ▶ Comme on l'a vu au cours précédent, d'une preuve de **False** (qui n'a aucun constructeur), on peut déduire n'importe quoi.
- ▶ `Lemma two_two_five : False -> 2 + 2 = 5.`  
`Proof. intros h. destruct h. Qed.`
- ▶ Idem, avec *intro pattern* (`intros` et `destruct` simultanés) :  
`Lemma foo : False -> 2 + 2 = 5. Proof. intros []. Qed.`
- ▶ On appelle ce principe *ex falso quod libet* (du latin pour « du faux, tout ce qu'on veut ») ou encore, « principe d'explosion ».
- ▶ En logique (intuitionniste), ceci correspond à la règle d'élimination de  $\perp$  (symbole pour **False**) :

$$\frac{\Gamma \vdash \perp}{\Gamma \vdash P} (\perp E)$$

# Élimination de `False` et terme de preuve

- ▶ Lorsqu'on définit :

```
Inductive False : Prop := .
```

- ▶ Rocq fournit lui-même des « principes d'induction » (qui diffèrent par les sortes, mais peu importe...), dont

```
Check False_ind. (* forall P : Prop, False -> P. *)
```

- ▶ Et en fait :

```
Print False_ind.
(* = fun (P : Prop) (f : False)
 => match f return P with end.
*)
```

- ▶ Ce `match` étrange a le type `P` pour n'importe quel `P`, car toutes les branches (il n'y en a aucune) ont ce type.